

Proyecto final — Bases de datos

Reparto detallado de actividades por integrante · Equipo de 6 · Plazo: menos de 2 semanas

Estrategia general

Dividimos el trabajo en seis bloques temáticos. Cada integrante lidera un bloque y es responsable de sus entregables, pero todos participan en el diseño inicial y en las revisiones cruzadas. Así nadie queda aislado en su parte y todos podemos defender cualquier punto en la exposición.

El proyecto tiene **dos partes**: la primera es el diseño e implementación de la base de datos del restaurante en PostgreSQL; la segunda es un script que orquesta la carga de archivos de texto hacia otra BD. Más todo lo correspondiente a documentación formal en LaTeX, capa de conexión y exposición.

Fase 0 — Trabajo conjunto (días 1-2)

Antes de repartir tareas individuales, todos hacemos juntos las primeras decisiones de diseño. Esto es innegociable: si una sola persona arma el MER, el resto va perdido toda la exposición.

Sesión 1 — Análisis del enunciado (1 hora)

- Leer el enunciado completo en voz alta, párrafo por párrafo.
- Identificar entidades, atributos, relaciones y reglas de negocio explícitas.
- Listar los **7 requerimientos funcionales** que pide el enunciado (triggers, vistas, funciones, índice).
- Aclarar dudas sobre cardinalidades, opcionalidad y atributos compuestos (domicilio, nombre).

Sesión 2 — Construcción del MER (2-3 horas)

- Dibujar el MER en drawio o Lucidchart, todos viendo la misma pantalla.
- Decisiones de diseño ya tomadas:
 - **EMPLEADO** con herencia ISA hacia Cocinero, Mesero y Administrativo (con solapamiento permitido).
 - **DEPENDIENTE** como entidad débil de EMPLEADO (CURP, nombre, parentesco).
 - **FACTURA NO existe como entidad**; se modela como relación *facturada_a* entre ORDEN y CLIENTE.
 - **fecha_hora** como un solo atributo TIMESTAMP en ORDEN (no fecha + hora por separado).
 - Atributo **subtotal** en la relación *contiene* (no *precio_total*).
 - Atributo **foto** en EMPLEADO.
 - PRODUCTO incluye atributo **receta**.
- Validar que todo el enunciado esté cubierto antes de cerrar el MER.

Sesión 3 — MR y normalización (1-2 horas)

- Convertir el MER en MR (modelo relacional): tablas, llaves primarias, llaves foráneas.
- Verificar que todas las tablas estén en 3FN como mínimo.
- Descomponer atributos compuestos: *domicilio* → estado, CP, colonia, calle, número; *nombre* → nombre_pila, apellido_paterno, apellido_materno.
- Cerrada esta sesión, el integrante 1 ya puede empezar el DDL en paralelo.

Setup técnico (día 1 también, en paralelo)

- Crear el repositorio en **GitHub** con la estructura de carpetas que pide el enunciado.

- Cada quien con su rama personal (*feature/nombre-bloque*); merge a *main* vía pull request.
- Instalar PostgreSQL local + pgAdmin o DBeaver para todos.
- Levantar una base de datos compartida en la nube (Neon, Supabase o Railway tienen tier gratis) para que todos puedan probar contra el mismo entorno.
- Crear documento compartido en Overleaf para el LaTeX (lo lidera el integrante 6, pero todos pueden contribuir).

Fase 1 — Reparto detallado por integrante (días 3-10)

Cada bloque detalla: qué hace, qué entregables produce, en qué archivos del repo trabaja, y de qué depende.

Carlos

Arquitecto de datos

Diseño físico y DDL

Responsabilidades

- Traducir el MR al script DDL completo en PostgreSQL.
- Crear todas las tablas con tipos de datos correctos, llaves primarias, llaves foráneas, NOT NULL, CHECK constraints y valores por defecto.
- Implementar la herencia EMPLEADO → Cocinero/Mesero/Administrativo (recomendación: usar tabla por subclase con FK al empleado padre, o tabla única con discriminador; justificar la decisión).
- Implementar DEPENDIENTE como tabla con llave compuesta (CURP + num_empleado) y FK con *ON DELETE CASCADE*.
- Definir el dominio del atributo **folio** como VARCHAR con el formato ORD-XXX (la generación automática la hace el integrante 2 con secuencia).
- Crear la **secuencia** base para el folio (*CREATE SEQUENCE folio_seq*).
- Coordinar con todos para que sus scripts (triggers, funciones, vistas) se ejecuten DESPUÉS del DDL sin errores.

Entregables

- **01_ddl_creacion_tablas.sql** — script idempotente con DROP IF EXISTS y CREATE TABLE de todas las tablas en orden correcto.
- **02_secuencias.sql** — secuencias necesarias (folio_seq, num_empleado_seq si aplica).
- Diagrama del modelo físico (puede ser una exportación del drawio o un screenshot del DBeaver).
- Sección de *Diseño* en el documento LaTeX explicando decisiones (tipos de datos, restricciones, justificación de la herencia).

Dependencias

Bloquea a: todos los demás integrantes (sin DDL no hay tablas donde correr triggers, vistas ni cargas). Debe tener el DDL funcional al final del **día 4** como máximo.

Yukio

Especialista en lógica de negocio

Triggers + secuencias + reglas

Responsabilidades

- Implementar el **trigger de actualización de totales**: al insertar/actualizar una fila en *contiene*, debe (a) calcular el subtotal = cantidad × precio del producto; (b) validar que el producto esté disponible (lanzar EXCEPTION si no); (c) recalculer el total de la orden sumando todos sus subtotales.
- Implementar el **trigger o función de generación de folio**: cada nueva orden debe recibir un folio con formato ORD-001, ORD-002, etc., usando la secuencia que creó Carlos.
- Considerar también el caso de **UPDATE** y **DELETE** en *contiene*: si cambia la cantidad o se elimina un producto de la orden, el total debe recalcularse.
- Documentar el comportamiento esperado de cada trigger con ejemplos comentados.

Entregables

- **03_triggers.sql** — todos los triggers y sus funciones asociadas.
- **04_funcion_folio.sql** — función que genera el folio formateado a partir de la secuencia.
- Pruebas: un script **test_triggers.sql** con casos que demuestren que los triggers funcionan (insert, update, delete; producto disponible vs no disponible).
- Sección de *Implementación* — *triggers* en el documento LaTeX explicando el funcionamiento.

Dependencias

Depende de: Carlos (necesita las tablas creadas y la secuencia). Puede empezar a escribir el código en seco desde el día 1, pero solo puede probarlo a partir del día 4.

Evans

Especialista en consultas paramétricas

Stored procedures y funciones

Responsabilidades

- Implementar la **función de órdenes por mesero en el día**: recibe *num_empleado* como parámetro, valida que sea mesero (si no, lanza un mensaje de error con RAISE EXCEPTION), y devuelve el número de órdenes registradas hoy más el total cobrado.
- Implementar la **función de ventas por rango de fechas**: recibe *fecha_inicio* y *fecha_fin* (con *fecha_fin* opcional; si no se da, usa solo *fecha_inicio* como día único), y devuelve el número total de ventas y el monto total.
- Las funciones deben retornar tipos compuestos (RECORD o TABLE) para que devuelvan ambos valores en una sola llamada.
- Manejo de casos borde: ningún resultado, fechas inválidas, *num_empleado* inexistente.

Entregables

- **05_funciones_consulta.sql** — las dos funciones con sus tipos de retorno y comentarios.
- Script de prueba **test_funciones.sql** con llamadas de ejemplo y resultados esperados.
- Sección de *Implementación* — *funciones* en el documento LaTeX explicando inputs, outputs y validaciones.

Dependencias

Depende de: Carlos (necesita tablas) e idealmente Yukio (para probar con datos reales generados por triggers). Puede escribir el código en paralelo desde el día 3.

Responsabilidades

- Crear la **vista del platillo más vendido**: agrupa *contiene* por producto, suma cantidades, ordena descendente y muestra *todos los detalles* del producto top.
- Crear la **vista tipo factura**: une ORDEN + CLIENTE + CONTIENE + PRODUCTO para mostrar la información de facturación de cada orden (filtra solo las que tienen rfc_cliente NOT NULL). Considerar partirla en dos vistas si conviene: encabezado y detalle.
- Crear la consulta o vista de **productos no disponibles** que devuelve el nombre de los productos con *disponibilidad = false*.
- Diseñar y crear al menos un **índice** con justificación. Sugerencia: índice B-tree sobre *orden(fecha_hora)* para acelerar las consultas de ventas por rango (que usa Evans); o índice sobre *contiene(id_producto)* para acelerar la vista del platillo más vendido.
- Justificar por escrito: *qué tipo de índice eligieron, sobre qué columna, y por qué*.

Entregables

- **06_vistas.sql** — todas las vistas con CREATE VIEW y comentarios.
- **07_indices.sql** — el o los índices creados.
- Pruebas: queries que usen las vistas y muestren resultados.
- Sección de *Implementación* — *vistas e índices* en el LaTeX, incluyendo la **justificación del índice** (tipo, columna, motivo).

Dependencias

Depende de: Carlos (DDL). Para probar las vistas necesita datos, así que conviene coordinar con Ñañez que se encarga de la carga inicial.

Responsabilidades

- Diseñar el formato de los **archivos de texto de entrada** (CSV o similar) para cada tabla: empleados, productos, clientes, órdenes, contiene, etc. Acordar con el equipo qué columnas y qué separador.
- Crear una **segunda base de datos** destino (puede ser otra BD en el mismo servidor PostgreSQL).
- Construir un script de **orquestación** que lea los archivos en orden correcto (respetando FKs: primero empleados, luego órdenes, etc.) y los inserte en la BD destino.
- Implementar **validaciones de flujo**: archivo inexistente, columnas faltantes, tipos de datos inválidos, violaciones de FK. En caso de error, hacer rollback parcial o registrar en log y continuar (decidir política con el equipo).
- Generar un **archivo de log** con el resumen de cada ejecución: registros insertados, registros rechazados, errores encontrados.
- Tecnología recomendada: Python con *psycopg2* y manejo de transacciones. Alternativa: bash con *psql* + *COPY*.
- Generar archivos de texto de prueba con datos sintéticos para demostrar el funcionamiento.

Entregables

- **parte2/cargar_datos.py** (o equivalente) — script principal con manejo de errores.
- **parte2/archivos_ejemplo/** — carpeta con CSVs de muestra para cada tabla.
- **parte2/README.md** — instrucciones de uso del script.
- Sección de la Parte 2 en el documento LaTeX explicando arquitectura, decisiones y manejo de errores.

Dependencias

Depende de: Carlos (necesita el DDL para replicarlo en la BD destino). Puede empezar el día 3 una vez definido el esquema.

Héctor

Documentación + presentación + conexión

LaTeX, exposición y capa de aplicación

Responsabilidades

- Construir el **documento formal en LaTeX** con todas las secciones que pide el enunciado: Introducción, Plan de trabajo, Diseño, Implementación, Presentación, Conclusiones.
- Es **editor del documento**, no autor único: los demás integrantes aportan los textos de sus secciones técnicas; el integrante 6 los integra, da formato, cuida ortografía y consistencia.
- Construir la **capa de conexión** a la BD: una mini aplicación que se conecte a PostgreSQL y demuestre funcionalidad (puede ser una CLI simple en Python, una página web ligera con Flask, o una app de escritorio con Tkinter). El objetivo es *presentar la solución como software al cliente* según pide el enunciado.
- La app debe poder al menos: agregar productos a una orden, consultar ventas del día, mostrar la factura de una orden, ver el platillo más vendido. (Es decir, exponer visualmente los puntos del enunciado).
- Preparar la **presentación** (Beamer o PowerPoint formal) con la estructura de exposición acordada.
- Coordinar los **ensayos del día 14**.

Entregables

- **documentacion/Proyecto.tex** + PDF compilado.
- **app_conexion/** — código fuente de la capa de conexión.
- **presentacion/Exposicion.pdf** — slides finales.
- Sección de *Presentación* en el LaTeX explicando qué hace la app y cómo se conecta a la BD.

Dependencias

Depende de: todos. El documento LaTeX se va llenando conforme los demás avanzan; la app de conexión necesita que las funciones y vistas estén listas para invocarlas. **Si termina antes**, apoya a Ñañez con scripts auxiliares.

Tabla resumen — Archivos en el repositorio

Estructura recomendada del repo. Cada archivo tiene un dueño claro, lo cual también sirve como evidencia del trabajo individual que pide el enunciado.

Archivo / Carpeta	Dueño	Contenido
01_ddl_creacion_tablas.sql	Carlos	CREATE TABLE de todo el esquema
02_secuencias.sql	Carlos	Secuencias (folio_seq, etc.)
03_triggers.sql	Yukio	Triggers de totales y validación
04_funcion_folio.sql	Yukio	Función para formato ORD-XXX
05_funciones_consulta.sql	Evans	Funciones de órdenes por mesero y ventas
06_vistas.sql	Eydan	Vista factura, platillo más vendido, no disponibles
07_indices.sql	Eydan	Índices con justificación
08_datos_prueba.sql	Equipo	INSERTs de datos demo (todos contribuyen)
parte2/	Ñañez	Script de carga y archivos de prueba
app_conexion/	Héctor	Capa de conexión / mini app
documentacion/	Héctor	LaTeX, PDF, MER, MR, presentación

Cronograma día por día

Día	Actividad	Responsable(s)
1	Análisis del enunciado + setup de GitHub y BD compartida	Todos
1-2	Diseño del MER conjunto	Todos
2	Paso a MR + decisiones de normalización	Todos
3-4	DDL completo (tablas, FKs, secuencias)	Carlos
3-5	Diseño del esquema de archivos de carga (Parte 2)	Ñañez
4-7	Triggers y función de folio	Yukio
4-7	Funciones de consulta (mesero, ventas por rango)	Evans
5-8	Vistas e índices	Eydan
5-9	Script de carga + validaciones + logs	Ñañez
6-10	Capa de conexión / mini app	Héctor
3-10	Avance continuo del LaTeX (cada quien aporta su sección)	Todos, lidera Héctor
11	Integración: correr todo el flujo de punta a punta	Todos
11-12	Code review: cada quien explica su bloque al grupo	Todos

12-13	Pruebas finales + ajustes + cierre del documento	Todos
13	Construcción de la presentación final	Héctor + equipo
14	Ensayos de exposición (cada quien expone una parte ajena)	Todos

Fase 2 — Integración y revisión cruzada (días 11-13)

Aquí está el truco para que todos aprendan parejo y nadie pueda quedar mal en la exposición.

Sesión de integración (día 11)

- Todos juntos, corren los scripts en orden: DDL → secuencias → triggers → funciones → vistas → índices → datos prueba.
- Si algo falla, se identifica el dueño y se corrige al momento.
- Se prueba el script de la Parte 2 contra la BD destino.
- Se prueba la app de conexión contra la BD principal.

Code reviews (días 11-12)

- Dos sesiones de 2 horas. En cada una, tres integrantes presentan su bloque al resto.
- Cada presentación es de 20 minutos: explicación + demo + preguntas.
- Los que escuchan toman notas; al final cada quien debe poder explicar TODOS los bloques.

Ensayo de exposición (día 14)

- Cada integrante expone una parte que **NO** sea la suya. Por ejemplo, el de triggers explica las vistas; el de documentación explica el DDL.
- Si alguien se traba, regresamos a estudiar antes de la presentación oficial.
- Esto nos blindamos contra cualquier pregunta cruzada del profesor.

Consideraciones importantes

Sobre el uso de IA

El enunciado dice explícitamente: *“Equipos que presenten códigos obtenidos de herramientas de IA o sitios web, no le será contado el punto del proyecto donde se identifique esta situación.”* Esto aplica para SQL, LaTeX, Python, todo. Podemos usar IA para entender conceptos o debuggear, pero el código que entregamos debe estar escrito y comprendido por nosotros. Recomendación: cada quien comenta su propio código con sus palabras, y todos podemos justificar línea por línea lo que entregamos.

Sobre GitHub y evidencia individual

El enunciado pide en el plan de trabajo un *“breve detalle de qué hizo cada miembro del equipo.”* Trabajar con ramas y pull requests genera esa evidencia automáticamente: los commits muestran exactamente qué aportó cada quien. No suban commits enormes; mejor varios commits pequeños con mensajes claros.

Sobre la presentación formal

El enunciado insiste en que la presentación debe ser *totalmente formal*, simulando una propuesta de software a un cliente. Esto significa: vestimenta formal, slides limpios (sin memes), demo funcional preparada y respaldo por si falla algo en vivo (screenshots o video de la demo).

Política ante imprevistos

- Si alguien se atrasa más de 24 horas en su entregable, el equipo se entera por el grupo de WhatsApp/Discord y se reorganiza.
- Daily standup corto (5-10 min) cada noche para reportar avances y bloqueos.

- Backup: Héctor, una vez terminada la app de conexión, puede ayudar a quien venga más cargado.